



Database Forensics

How Data Solves Crimes in Large Organisations

**A 7-slide guide to investigating data
at scale with SQL/ and audit logs**

EVODIA UNJUEH AKU
unjuehevo@gmail.com

Day 4 of 30 | June 2026





DATABASE FORENSICS = DIGITAL EVIDENCE

FORENSIC RECONSTRUCTION OF DATABASE

PART ONE

A company found a \$12,000 overpayment. The payroll shows no anomaly. HR said, “ Nothing was deleted.”

Rebuilding the truth from deleted data. (**How we recover** what **happened**, when, and **by** who - even after deletion)

B. TRANSACTION LOGS

1. The DB's CCTV Footage
2. Transaction log shows you the crime scene.
3. It's not a table. It's a sequential file (.ldf) in SQL Server

BANK TELLER'S NOTE BOOK

- RULE **1**: write before changing money
- RULE **2**: Never erase old lines
- RULE **3**: Every action = **1** new line

SQL Server does same thing.

Every DELETE, UPDATE, TRUNCATE = **1** log record

This is **WRITE-AHEAD LOGGING(WAL)** =
FOUNDATION OF FORENSICS

QUESTION: If the record is gone from the UI where do we find it?
INTERROGATE THE TRANSACTION LOG



TRANSACTION LOG

file(.ldf) → Every transaction is logged into this file before it is inserted into the database



TRANSACTION SYSTEM

Core engine that process data changes and relies on logs to maintain data integrity and consistency.

THE LIFE CYCLE

1. The request
2. Memory first(RAM)buffer pool
3. The log sequential write
4. The checkpoint.

Job: Make DB “ACID compliant”. Every change = **1** log recorded before data changes (Atomicity, Consistency, Isolation, Durability.) REDO completed work or UNDO half back changes, prevent data corruption.

Analogy: It’s like CCTV + Receipt. Before SQL updates your bank balance, it writes: “Old = 1000, New = 900, Time = 10: 32am, User = John”

RECOVERY MODEL

1. Simple: Dev, text
2. Full: Production fully logged. Regular backup and preserve the log chain
3. Bulk logged

A. **99.99%** Effective for Crash Recovery B. **0% to 50%** Effective for Forensics(If unmanaged)

C. **100%** Effective for Point-in-Time Recovey.

BLIND SPOT

1. The wrong recovery model
2. Log corruption
3. VLF Over load(Poor maintenance)
4. Malicious Intervention(clear, detach or delete the log files)



THREE THINGS EVERY LOG RECORD CONTAINS

1. **LSN** - Log Sequence Number = timestamp/ID of the action
2. **Operation** - LOP_DELETE, LOP_INSERT, LOP_MODIFY_ROW
3. **Before/After** Image - Old data vs New data.

UPDATE User SET SALARY = 5000 WHERE ID = 1

Log stores : Before: Salary = 3000 | After: Salary = 5000

ACTION	TABLE SHOWS	LOG SHOWS
DELETE	0 rows	"User X deleted ID 45 at 2:03am"
UPDATE	New values only	"Old = 3000, New = 5000"
TRUNCATE	Empty table	"All pages deallocated at 2:05am"

Attackers can wipe tables. They forget the log is write only CCTV.

TRANSACTION INFORMATION

1. All information needed to revert state
 - ☐ At least 1 entry per change
 - ☐ Update-statement internally DELETE/INSERT with linking
 - ☐ Timestamps
 - ☐ Metadata
 - ☐ Overwritten information
2. DELETE-statement only 'mark as delete'
3. Log types

HOW TO MAKE THE RECOVERY PROCESS EFFECTIVE?

1. Separate the Physical Storage(Isolate the Risk) (**.ldf**) not in the same physical drive array as (**.mdf**) data files.
2. Implement Regular Log Backups(running transaction log backups every 15min or even 5min for critical financial systems.
3. Size your Logs Correctly from the Start.
4. Lock Down Log Modifications.



LOG STRUCTURE

- VLF:** Virtual Log File. Log file split into chunks. circular reuse
- LSN:** LOG SEQUENCE NUMBER: 3-part address: **VLF:Block:Slot**. Unique ID for every action
- Log record:** 1 action = 1 record. Never update, only appended
- KEY:** Old records get overwritten when VLF “inactive”. **Don’t wait to collect evidence**

EVERY LOG RECORD CONTAINS - THE 7 FORENSIC FIELDS

- LSN:** Where/when in log sequence
- XACT ID:** Transaction ID. Groups all actions by 1 query
- Operation:** LOP_INSERT, LOP_DELETE, LOP_MODIFY_ROW, LOP_BEGIN_XACT
- AllocUnit:** Which table/index was hit
- Transaction SID:** Who did it. convert with **SUSER_SNAME()**
- Begin Time:** Timestamp of transaction start
- Before/After Image:** Rowlog Contents 0 = old data, RowLog Contents 1 = new data

TWO TYPES:

- ACTIVE LOG-**(Still running or haven’t been safely written down to the main data file. This can never be overwritten
- INACTIVE LOG-**Contains old transactions that are finished and finalized

PART	ROLE IN CRIME	FORENSIC VALUE
LOP_BEGIN_XACT	Scene opens	Who + When attack started
LOP_INSERT/DELETE/UPDATE	The attack	What data was changed
LOP_COMMIT_XACT	Scene closes + saves	Prove intent, not accident
LOP_ABORT	Attack failed	Shows attempted fraud
VLF_Truncation	CCTV tape reuse	Explains missing evidence
TRUNCATE/PAGE ops	Mass deletion tool	proves attacker used bulk tools

SSMS (SQL CODE)

-- Step 1: See if log exists

DBCC LOGINFO

-- Step 2: Read uncommitted log who did what

SELECT

[Current LSN], [Operation], [Transaction Name],

[Begin Time],[Transaction SID]) as Username,

[Rowlog Contents 0], [Rowlog Contents 1]

FROM fn_dblog(NULL,NULL)

WHERE [Operation] IN

('LOP_DELETE', 'LOP_INSERT', 'LOP_MODIFY_ROW')

ORDER BY [Current LSN] DESC

RUN: INSERT 1 row

UPDATE it

DELETE it.

Then run `fn_dblog`.

you will see that the log saw everything.

KEY TAKE AWAY

1. Table lie. Log testify
2. DELETE is not deletion, it is relocation
3. Every crime has three crime timestamps BEGIN time = planning.
4. Operation time = execution. COMMIT time = intent. All 3 log.
5. Bulk operation are attacker favorite
6. COMMIT proves intent in court
7. Speed beats skill
8. Absence of evidence is evidence

COMMAND	SCOPE OF ACTION	LOG SIGNATURE/LOG FOOTPRINT	STEALTH LEVEL	FORENSIC DEFINITION
UPDATE	Modifies specific values inside existing rows	LOP_MODIFY_ROW(Only captures the change, not the whole table)	MAXIMUM SILENT	HARDEST TO SPOT
DELETE	Removes specific rows row-by-row based on condition	LOP_DELETE_ROWS(Heavy footprint;logs every single deleted row)	LOW TRACEABLE	STRAIGHT FORWARD
TRUNCATE	Empties the table by dropping it's underlying data pages	LOP_MODIFY_HEADER(Light footprint;logs only page deallocation)	MEDIUM LOUD	INTERMEDIATE
DROP	Obliterate the entire table object from the database schema	LOP_DROP_DROP_DROP (Logs the destruction of the tables definition)	LOW CATASTROPHIC	IMMEDIATE DETECTION due to application crashes. Require reconstructing system metadata first.